Secure Message App

Advanced Network Security 6605 Extra Credit Project

A terminal-based secure client-server chat application using Python sockets

Student:	Sivananda Pranav Sarma Devaguptapu
Project Type:	Secure network application
Core Requirements:	Mutual authentication, confidentiality, message integrity
Implementation:	Python sockets and cryptography package
Demo Address:	127.0.0.1:5000

Supervising professor: Dr. Baek-Young Choi
Submission Report

Contents

Project Overview

This project implements a terminal-based secure client-server chat application using Python sockets. The application runs locally on `127.0.0.1:5000`. It demonstrates a complete educational security protocol in which the client and server authenticate each other, establish a shared AES session key, and exchange encrypted messages.

Assignment Requirements Mapping

Requirement	How the Project Satisfies It
Network application using sockets	The application uses Python TCP sockets in `server.py`, `client.py`, and shared socket helpers in `crypto_utils.py`.
Mutual authentication	The client and server each sign random nonce challenges using RSA-PSS signatures. Each side verifies the other side's signature using the corresponding public key.
Confidentiality	The client generates a random AES-256 session key. This session key is encrypted with the server's RSA public key using RSA-OAEP before being sent. Chat messages are encrypted with AES-256-GCM.
Message integrity	AES-GCM includes an authentication tag. If an encrypted packet is modified, decryption fails and the program reports an integrity check failure.
Design document with diagrams	The repository contains `DESIGN.md`, and this report includes both high-level and low-level protocol diagrams.
Working application evidence	The repository contains `DEMO_TRANSCRIPT.md`, and this report includes screenshots of key generation, server/client execution, encrypted messaging, and tamper detection.

Repository Structure

The final project folder is organized as follows:

```
secure-message-app/
|-- client.py
|-- server.py
|-- crypto_utils.py
|-- generate_keys.py
|-- tamper_demo.py
|-- requirements.txt
|-- README.md
|-- DESIGN.md
|-- DEMO_TRANSCRIPT.md
|-- .gitignore
|-- keys/
|   `-- .gitkeep
`-- screenshots/
    `-- .gitkeep
```

Note: Generated PEM key files are not committed to the repository. They are created locally by running `python generate_keys.py`.

Security Protocol Design

Security Goals

Mutual authentication. The client and server each prove ownership of their private keys by signing unpredictable nonce challenges. This prevents an unauthenticated party from pretending to be either side.

Confidentiality. The AES session key is protected during transport using RSA-OAEP. After the session key is established, chat messages are encrypted using AES-256-GCM, so plaintext is not sent directly across the socket.

Message integrity. AES-GCM authenticates each encrypted packet. If ciphertext is modified, the authentication tag does not verify and decryption fails.

High-Level Protocol Diagram

```
Client                                  Server
  | -------- TCP connect -------------> |
  | <------- server nonce ------------- |
  | ---- sign(server nonce) ----------> |
  | ---- client nonce ----------------> |
  | <--- sign(client nonce) ----------- |
  | ---- RSA-OAEP(AES key) -----------> |
  | ===== AES-GCM encrypted chat ===== |
```

Low-Level Cryptographic Flow

```
1. Server creates server_nonce
2. Client signs server_nonce with client_private.pem
3. Server verifies signature with client_public.pem

4. Client creates client_nonce
5. Server signs client_nonce with server_private.pem
6. Client verifies signature with server_public.pem

7. Client creates random 32-byte AES session key
8. Client encrypts AES key with server_public.pem using RSA-OAEP
9. Server decrypts AES key with server_private.pem

10. Each chat message:
    plaintext
      -> AES-256-GCM encryption with random 12-byte nonce
      -> packet = nonce || ciphertext || authentication tag
      -> 4-byte length prefix + packet over TCP socket
```

Step-by-Step Operation

1. The server starts listening on `127.0.0.1:5000`.

2. The client opens a TCP socket connection to the server.

3. The server sends a random 32-byte nonce challenge to the client.
4. The client signs the server nonce using `client_private.pem` with RSA-PSS and SHA-256.
5. The server verifies the client signature using `client_public.pem`.
6. The client sends its own random 32-byte nonce challenge to the server.
7. The server signs the client nonce using `server_private.pem` with RSA-PSS and SHA-256.
8. The client verifies the server signature using `server_public.pem`.
9. The client generates a random AES-256 session key.
10. The client encrypts the AES key using the server's RSA public key with RSA-OAEP.
11. The server decrypts the AES key using its RSA private key.
12. Both sides exchange chat messages using AES-256-GCM encrypted packets.

Cryptographic Choices

- **RSA-PSS signatures:** Used for authentication because they allow each side to prove possession of a private key by signing a fresh random nonce.
- **RSA-OAEP key transport:** Used to protect the AES session key while it is sent from the client to the server.
- **AES-256-GCM:** Used for chat messages because it provides both encryption and integrity checking through an authentication tag.
- **4-byte length-prefix framing:** Used so the receiver can correctly determine message boundaries over TCP.

Application Demo

This section documents the working application using screenshots captured from the final project folder at `D:/Projects/6605_Spring_26/secure-message-app`.

Key Generation

Before running the client and server, the RSA keys are generated locally using `python generate_keys.py`. The generated PEM files are used by the client and server but are ignored by Git.

```
PS D:\Projects\6605_Spring_26\secure-message-app> ls


    Directory: D:\Projects\6605_Spring_26\secure-message-app


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        16-05-2026     18:36                keys
d-----        16-05-2026     18:29                screenshots
d-----        16-05-2026     18:40                __pycache__
-a----        16-05-2026     18:21             59 .gitignore
-a----        13-05-2026     18:25           4059 client.py
-a----        13-05-2026     18:09           3779 crypto_utils.py
-a----        16-05-2026     18:21           2466 DEMO_TRANSCRIPT.md
-a----        16-05-2026     18:21           4727 DESIGN.md
-a----        13-05-2026     18:09           1455 generate_keys.py
-a----        16-05-2026     18:21           2812 README.md
-a----        13-05-2026     18:16             13 requirements.txt
-a----        13-05-2026     18:25           4557 server.py
-a----        13-05-2026     18:21           1260 tamper_demo.py


PS D:\Projects\6605_Spring_26\secure-message-app> python generate_keys.py
Generated RSA keys in the keys/ folder.
PS D:\Projects\6605_Spring_26\secure-message-app>
```

Figure 1: Key generation and project directory contents.

Server Startup

The server is started first. It listens on `127.0.0.1:5000` and waits for a client connection.

```
PS D:\Projects\6605_Spring_26\secure-message-app> python server.py
Server listening on 127.0.0.1:5000...
```

Figure 2: Server listening on localhost port 5000.

Client Authentication and Session Establishment

The client connects to the server and completes the mutual authentication protocol. The screenshot shows that the client receives the server nonce, signs it, sends its own nonce, verifies the server signature, generates the AES session key, and establishes the secure session.

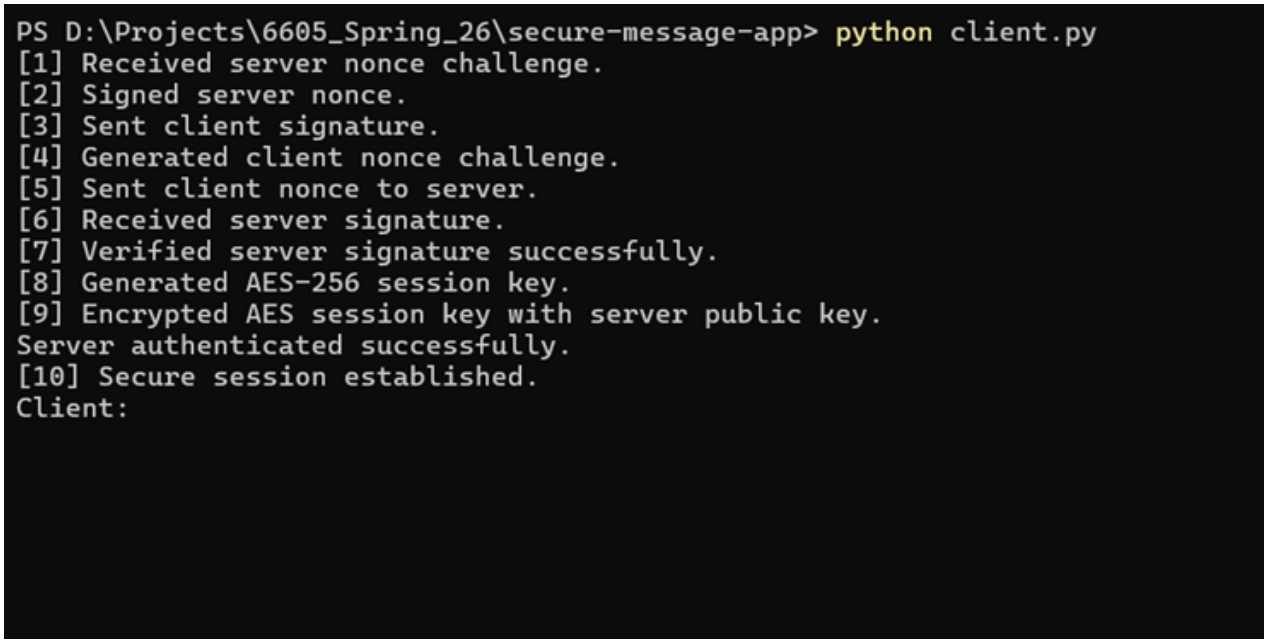

Figure 3: Client-side mutual authentication and secure session establishment.

Encrypted Message from Client

After the secure session is established, the client sends a message. The encrypted packet preview shows ciphertext, demonstrating that plaintext is not directly sent over the socket.

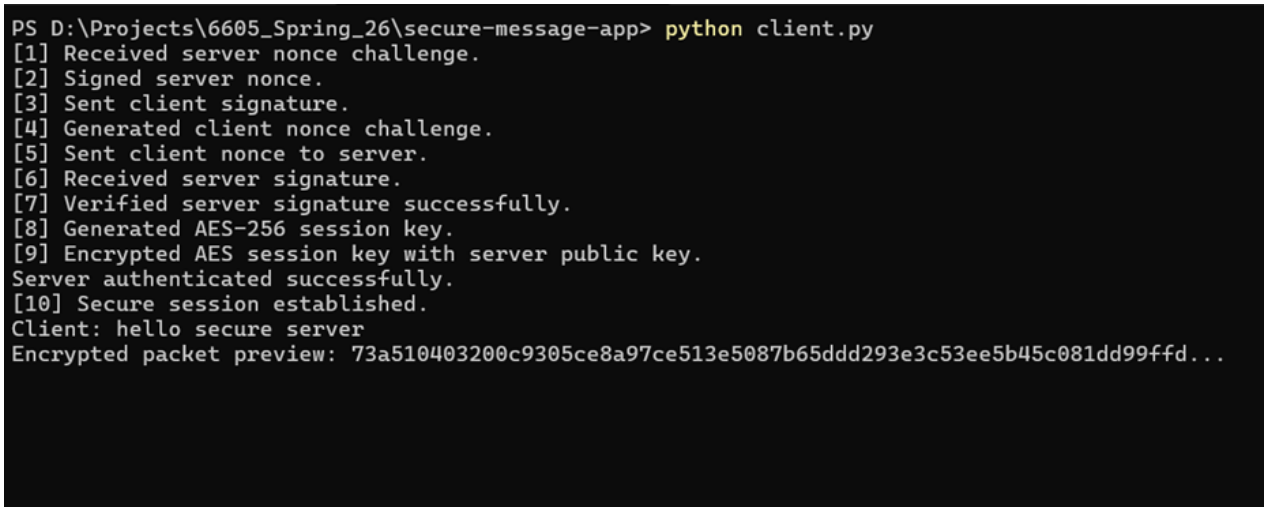

Figure 4: Client sends a message and prints an encrypted packet preview.

Encrypted Reply from Server

The server receives the client's message and sends an encrypted reply. The server output also displays an encrypted packet preview.

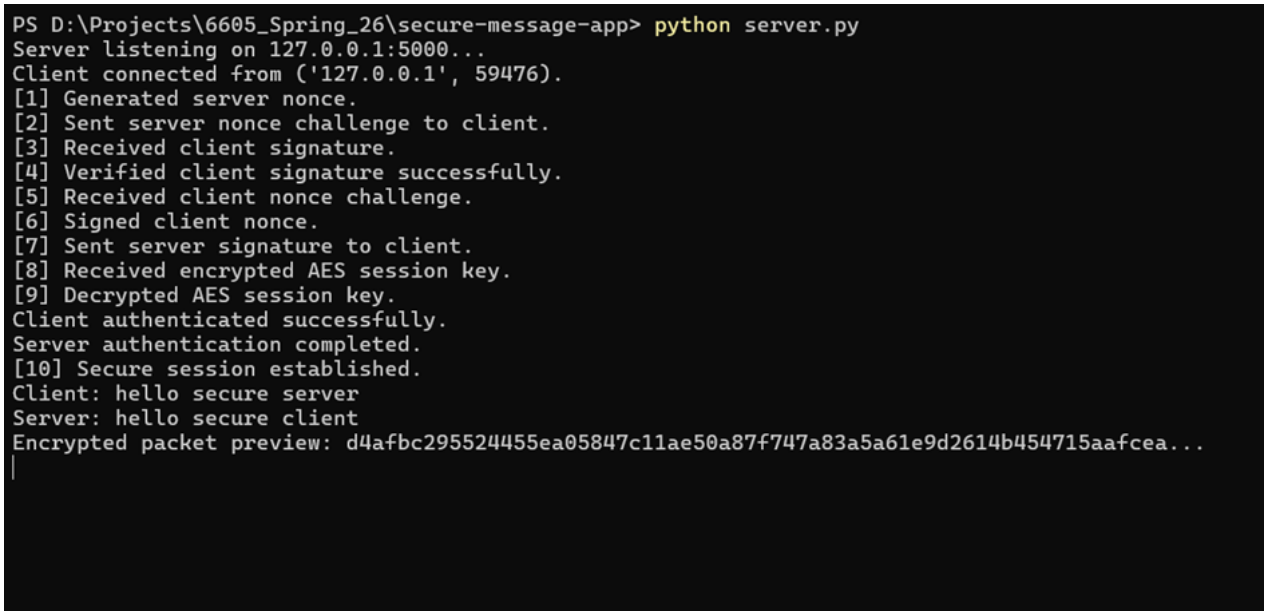

Figure 5: Server receives the client message and replies through the encrypted channel.

Clean Exit

The application can close cleanly after the chat is finished.

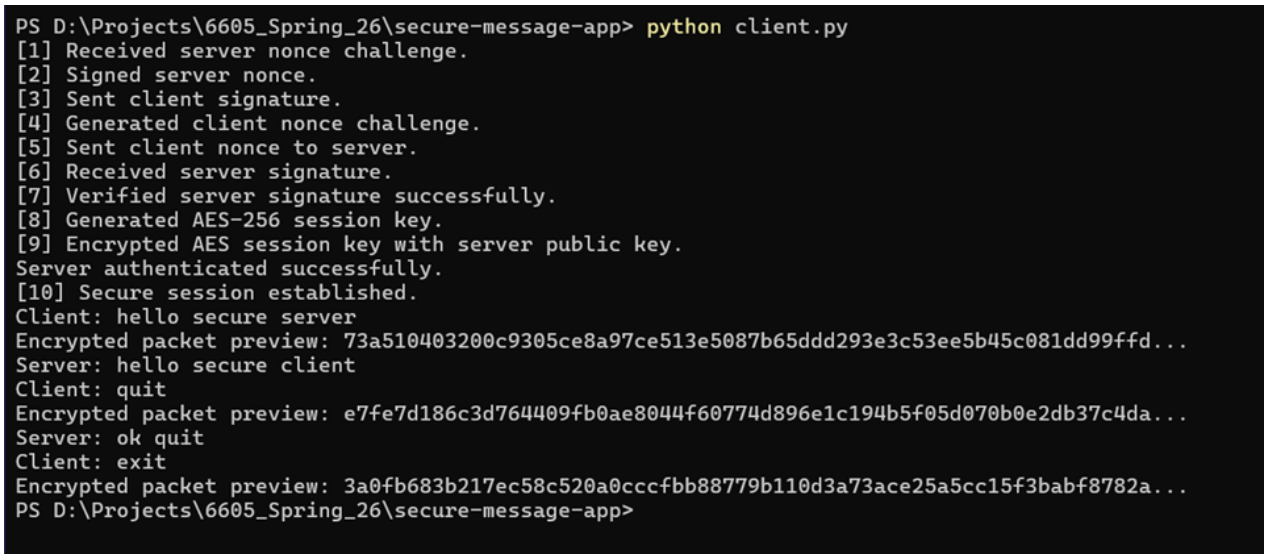

Figure 6: Client-side output showing encrypted messages and clean exit.

```
PS D:\Projects\6605_Spring_26\secure-message-app> python server.py
Server listening on 127.0.0.1:5000...
Client connected from ('127.0.0.1', 59476).
[1] Generated server nonce.
[2] Sent server nonce challenge to client.
[3] Received client signature.
[4] Verified client signature successfully.
[5] Received client nonce challenge.
[6] Signed client nonce.
[7] Sent server signature to client.
[8] Received encrypted AES session key.
[9] Decrypted AES session key.
Client authenticated successfully.
Server authentication completed.
[10] Secure session established.
Client: hello secure server
Server: hello secure client
Encrypted packet preview: d4afbc295524455ea05847c11ae50a87f747a83a5a61e9d2614b454715aafcea...
Client: quit
Server: ok quit
Encrypted packet preview: fa76864e7fb6ac9c7de55e1f53b089f42a282cb7317763c956dc715e83baf7bd...
Client: exit
PS D:\Projects\6605_Spring_26\secure-message-app>
```

Figure 7: Client-side output showing encrypted messages and clean exit.

Tamper and Integrity Failure Demo

The `tamper_demo.py` script encrypts a message, modifies one byte of the encrypted packet, and then attempts to decrypt it. AES-GCM rejects the tampered packet because the authentication tag no longer verifies.

```
PS D:\Projects\6605_Spring_26\secure-message-app> ls


    Directory: D:\Projects\6605_Spring_26\secure-message-app


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        16-05-2026     18:36                keys
d-----        16-05-2026     18:29                screenshots
d-----        16-05-2026     18:40                __pycache__
-a----        16-05-2026     18:21             59 .gitignore
-a----        13-05-2026     18:25           4059 client.py
-a----        13-05-2026     18:09           3779 crypto_utils.py
-a----        16-05-2026     18:21           2466 DEMO_TRANSCRIPT.md
-a----        16-05-2026     18:21           4727 DESIGN.md
-a----        13-05-2026     18:09           1455 generate_keys.py
-a----        16-05-2026     18:21           2812 README.md
-a----        13-05-2026     18:16             13 requirements.txt
-a----        13-05-2026     18:25           4557 server.py
-a----        13-05-2026     18:21           1260 tamper_demo.py


PS D:\Projects\6605_Spring_26\secure-message-app> python tamper_demo.py
Original plaintext: This message should stay private and unchanged.
Encrypted packet preview: a3dcd679977fd021d9ebb461d87e7ab4937df6ba9c6493ad9f3819b28e0c7dc5...
Tampered packet preview: a3dcd679977fd021d9ebb461d87e7ab4937df6ba9d6493ad9f3819b28e0c7dc5...
Decryption failed because message integrity check failed.
PS D:\Projects\6605_Spring_26\secure-message-app>
```

Figure 8: Tamper demo showing integrity-check failure after ciphertext modification.

Documentation Evidence

The project repository includes a concise `README.md`, a full `DESIGN.md`, and a `DEMO_TRANSCRIPT.md`. These files document how to run the project, how the protocol works, and what output is expected.

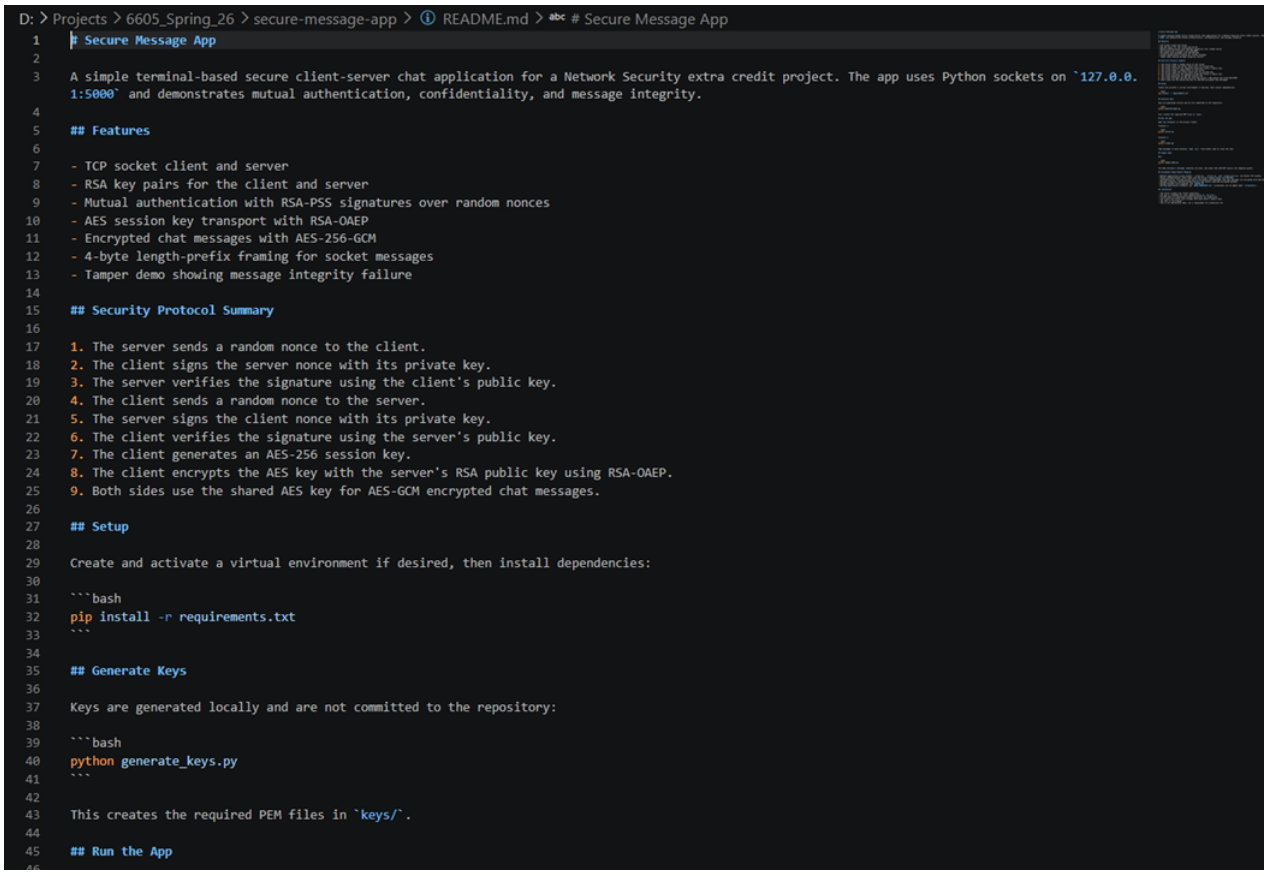

Figure 9: README file containing setup instructions and assignment requirement mapping.

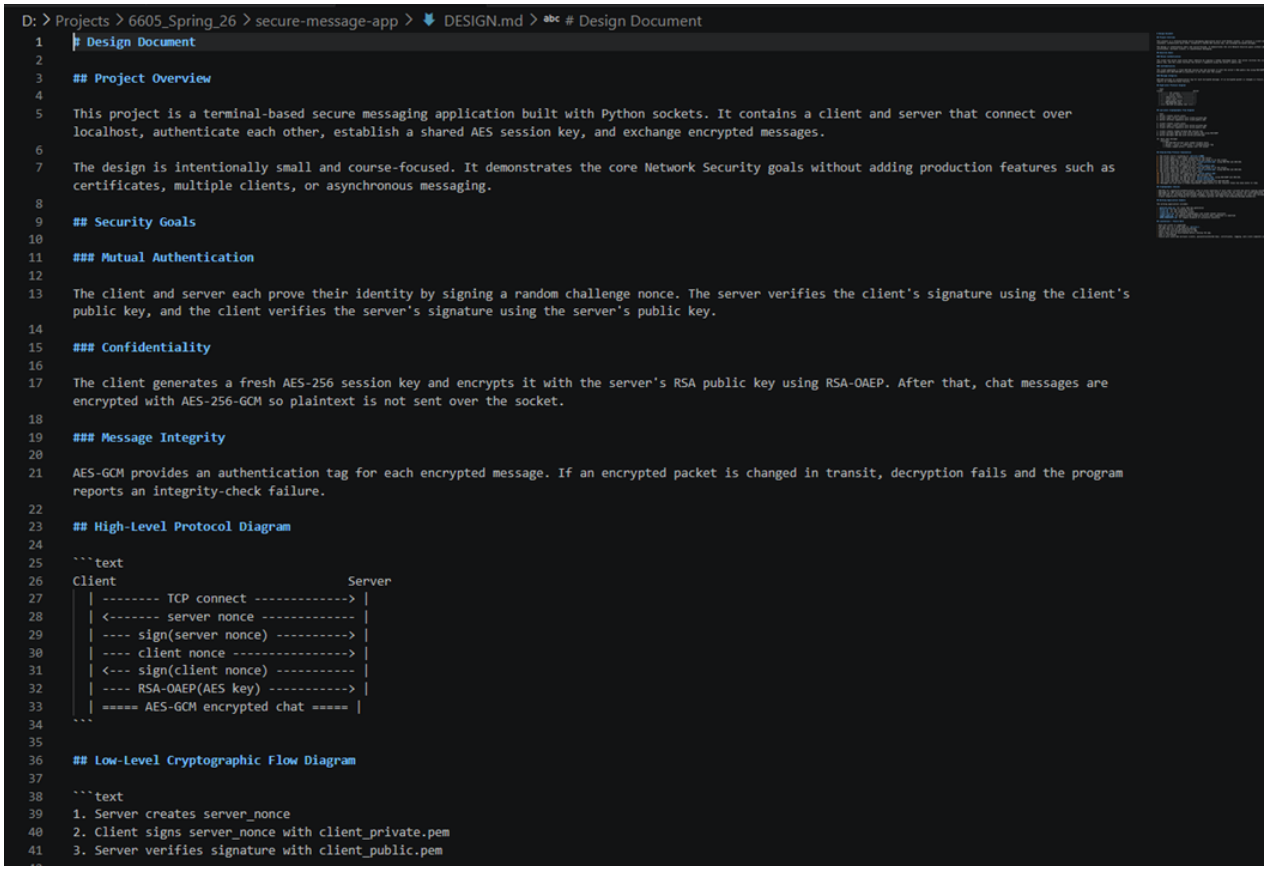

Figure 10: DESIGN document showing the protocol explanation and diagrams.

Limitations

This project is an educational demonstration and intentionally avoids production complexity. The main limitations are:

- The server accepts one client connection.
- The application is intended for local demonstration on `127.0.0.1`.
- Private keys are generated without passwords for simplicity.
- Public keys must already be available to both parties.
- There is no certificate authority or public key infrastructure.
- The chat is turn-based instead of fully asynchronous.
- The project is not intended to replace production TLS.

Conclusion

The Secure Message App successfully implements a socket-based secure communication protocol with mutual authentication, confidentiality, and message integrity. RSA-PSS signatures authenticate both the client and server, RSA-OAEP protects the AES session key during transport, and AES-256-GCM protects chat messages after the secure session is established. The screenshots and demo transcript show that the application runs correctly, exchanges encrypted messages, and detects tampered ciphertext.